



Lab Title: Web Application Reconnaissance

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS102 – Web Application Security Essential

Instructor Name: Ammar Aminu

Date of Submission: 28/12/2025

Executive Summary

This lab focused on practical web application reconnaissance and vulnerability discovery using a controlled, vulnerable environment. The assessment required setting up an attacker machine (Kali Linux) and a target machine (Metasploitable2) to simulate real-world web security testing scenarios. Through hands-on use of industry-standard tools, the lab covered technology fingerprinting, subdomain enumeration, directory discovery, database exploration, and SQL injection testing.

By performing each task step by step and capturing clear evidence, the lab demonstrated how attackers gather information and exploit weaknesses in web applications, as well as how defenders can understand and mitigate these risks. The exercises reinforced the importance of reconnaissance as the foundation of web application penetration testing and provided practical exposure to tools such as WhatWeb, Dirb, Sublist3r, KnockPy, MySQL Client, and Burp Suite.

Lab Objectives

The main objective of this lab was to develop hands-on skills in web application reconnaissance and vulnerability discovery. Specifically, the lab aimed to:

- Identify technologies used by a target web server
- Enumerate subdomains using multiple tools and compare their effectiveness
- Discover hidden directories and files on a web server
- Access and explore databases running on a vulnerable system
- Demonstrate first-order SQL injection under different security levels
- Understand how HTTP GET and POST requests differ
- Explain core web concepts such as information gathering, request–response flow, DNS, and the role of Burp Suite

Overall, the lab was designed to bridge theoretical knowledge with practical exploitation techniques commonly used in real-world web application security testing.

Tools Used

The following tools and platforms were used during the lab:

- **Kali Linux** – Attacker machine used for all reconnaissance and exploitation activities
- **Metasploitable2** – Vulnerable target machine hosting multiple insecure services
- **WhatWeb** – Used for web technology fingerprinting
- **Sublist3r** – Used for subdomain enumeration through OSINT sources

- **KnockPy** – Used for brute-force and reconnaissance-based subdomain discovery
- **Dirb** – Used for directory and file enumeration on the web server
- **MySQL Client** – Used to connect to and explore MySQL databases on Metasploitable
- **Burp Suite** – Used to intercept, modify, and analyze HTTP requests
- **Mutillidae** – Vulnerable web application used for SQL injection testing

Methodology

The lab was carried out using a structured, step-by-step methodology to ensure accurate results and proper documentation:

1. **Environment Setup**
Kali Linux and Metasploitable2 were configured on the same network. Network connectivity was verified by identifying the Metasploitable IP address and confirming communication between both machines.
2. **Technology Fingerprinting**
WhatWeb was used to perform both basic and aggressive scans against the Metasploitable web server to identify technologies, server details, and application frameworks in use.
3. **Subdomain Enumeration**
Sublist3r and KnockPy were run against the specified target domains. The results from both tools were collected and compared to evaluate coverage and effectiveness.
4. **Directory Enumeration**
Dirb was executed against the Metasploitable web server to discover hidden directories and files that could expose sensitive functionality or misconfigurations.
5. **Database Exploration**
The MySQL client was used to log into the MySQL service running on Metasploitable. Available databases were listed, and selected databases were explored by enumerating their tables.
6. **SQL Injection Testing**
Mutillidae was used to demonstrate first-order SQL injection. Tests were performed at low security through direct input manipulation and at higher security levels by intercepting and modifying requests using Burp Suite.
7. **Conceptual Analysis**
Additional sections focused on explaining GET vs POST requests, information gathering in web security, how websites work using the request–response model, the role of DNS, and the importance of Burp Suite in penetration testing.

Throughout the lab, screenshots of commands, outputs, and successful exploitation were captured as evidence, and all explanations were written in clear, original language to reflect personal understanding.

Part 1: Technology Fingerprinting with WhatWeb

Task

- Command used for basic scanning

```
(raph@kali)-[~]
$ whatweb http://192.168.146.129
http://192.168.146.129 [200 OK] Apache[2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2], IP[192.168.146.129], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux], WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
```

- Command used for aggressive scanning

```
(raph@kali)-[~]
$ whatweb --aggression 3 -v 192.168.146.129
[+] Simple hostname detected: 192.168.146.129. Testing both HTTP and HTTPS.
ERROR Opening: https://192.168.146.129 - Connection refused - connect(2) for "192.168.146.129" port 443
WhatWeb report for http://192.168.146.129
Status      : 200 OK
Title       : Metasploitable2 - Linux
IP          : 192.168.146.129
Country     : RESERVED, ZZ

Summary    : Apache[2.2.8], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2], PHP[5.2.4-2ubuntu5.10], WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

Detected Plugins:
[ Apache ]
The Apache HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows NT. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards.

Version      : 2.2.8 (from HTTP Server Header)
Google Dorks : (3)
Website      : http://httpd.apache.org/

[ HTTPServer ]
HTTP server header string. This plugin also attempts to identify the operating system from the server header.

OS           : Ubuntu Linux
```

- Short comparison of findings

Whatweb <http://192.168.146.129> - This command is used to quickly check which ports are open on a target. It is simple, fast, and commonly used at the beginning of a scan. While

Whatweb --aggressive 3 -v 192.168.146.129 - This command performs a detailed scan by checking the operating system, services, versions, and running scripts. It provides more information but takes longer and is easier to detect.

Part 2: Subdomain Enumeration and Tool Comparison

Task

Commands used

- icdfa.edu.ng



```
(laph@kali)-[~]
$ sublist3r -d icdfa.edu.ng

Sublist3r

# Coded By Ahmed Aboul-Ela - @aboul3la

[-] Enumerating subdomains now for icdfa.edu.ng
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[!] Error: Virustotal probably now is blocking our requests
```

- nasarawastate.gov.ng

```
(raph@kali)-[~]
$ sublist3r -d nasarawastate.gov.ng

# Coded By Ahmed Aboul-Ela - @aboul3la

[-] Enumerating subdomains now for nasarawastate.gov.ng
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSdumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[!] Error: Virustotal probably now is blocking our requests
Process DNSdumpster-8:
Traceback (most recent call last):
  File "/usr/lib/python3.13/multiprocessing/process.py", line 313, in _bootstrap
    self.run()
    ~~~~~^^
  File "/usr/lib/python3/dist-packages/sublist3r.py", line 269, in run
    domain_list = self.enumerate()
```

- icdfa.org.ng

```
(raph@kali)-[~]  
$ sublist3r -d icdfa.org.ng  
  
Sublist3r  
# Coded By Ahmed Aboul-Ela - @aboul3la  
  
[-] Enumerating subdomains now for icdfa.org.ng  
[-] Searching now in Baidu..  
[-] Searching now in Yahoo..  
[-] Searching now in Google..  
[-] Searching now in Bing..  
[-] Searching now in Ask..  
[-] Searching now in Netcraft..  
[-] Searching now in DNSdumpster..  
[-] Searching now in Virustotal..
```

- Output screenshots

icdfa.edu.ng screenshot output

```
IndexError: list index out of range
[-] Total Unique Subdomains Found: 32
apply.icdfa.edu.ng
www.apply.icdfa.edu.ng
campus.icdfa.edu.ng
www.campus.icdfa.edu.ng
caps.icdfa.edu.ng
www.caps.icdfa.edu.ng
cbt.icdfa.edu.ng
www.cbt.icdfa.edu.ng
ctf.icdfa.edu.ng
www.ctf.icdfa.edu.ng
cybersmart.icdfa.edu.ng
www.cybersmart.icdfa.edu.ng
fellowship.icdfa.edu.ng
www.fellowship.icdfa.edu.ng
ijcf.icdfa.edu.ng
www.ijcf.icdfa.edu.ng
intranet.icdfa.edu.ng
www.intranet.icdfa.edu.ng
orientation.icdfa.edu.ng
www.orientation.icdfa.edu.ng
ortn.icdfa.edu.ng
www.ortn.icdfa.edu.ng
payment.icdfa.edu.ng
www.payment.icdfa.edu.ng
pcep.icdfa.edu.ng
www.pcep.icdfa.edu.ng
portal.icdfa.edu.ng
www.portal.icdfa.edu.ng
www.rc.icdfa.edu.ng
```

nasarawastate.gov.ng screenshot output

[-] Total Unique Subdomains Found: 108

www.nasarawastate.gov.ng
asat2019.nasarawastate.gov.ng
www.asat2019.nasarawastate.gov.ng
asset.nasarawastate.gov.ng
www.asset.nasarawastate.gov.ng
backup.nasarawastate.gov.ng
www.backup.nasarawastate.gov.ng
bbb.nasarawastate.gov.ng
www.bbb.nasarawastate.gov.ng
bck.nasarawastate.gov.ng
www.bck.nasarawastate.gov.ng
bict.nasarawastate.gov.ng
www.bict.nasarawastate.gov.ng
bin.nasarawastate.gov.ng
www.bin.nasarawastate.gov.ng
bpp.nasarawastate.gov.ng
www.bpp.nasarawastate.gov.ng
contracts.nasarawastate.gov.ng
www.contracts.nasarawastate.gov.ng
covid19.nasarawastate.gov.ng
www.covid19.nasarawastate.gov.ng
cpanel.nasarawastate.gov.ng
cpcalendars.nasarawastate.gov.ng
cpcontacts.nasarawastate.gov.ng
csc.nasarawastate.gov.ng
www.csc.nasarawastate.gov.ng
dms.nasarawastate.gov.ng
www.dms.nasarawastate.gov.ng
emporium.nasarawastate.gov.ng
www.emporium.nasarawastate.gov.ng
export.nasarawastate.gov.ng

lcdfa.org.ng screenshot ouput

```
[~] Total Unique Subdomains Found: 37
```

```
www.icdfa.org.ng  
academy.icdfa.org.ng  
www.academy.icdfa.org.ng  
alumni.icdfa.org.ng  
www.alumni.icdfa.org.ng  
api.icdfa.org.ng  
www.api.icdfa.org.ng  
apply.icdfa.org.ng  
www.apply.icdfa.org.ng  
ccpa.icdfa.org.ng  
www.ccpa.icdfa.org.ng  
cpanel.icdfa.org.ng  
cpcalendars.icdfa.org.ng  
cpcontacts.icdfa.org.ng  
ctf.icdfa.org.ng  
www.ctf.icdfa.org.ng  
fellowship.icdfa.org.ng  
www.fellowship.icdfa.org.ng  
iconicproject.icdfa.org.ng  
www.iconicproject.icdfa.org.ng  
mail.icdfa.org.ng  
icdfa.edu.ng.icdfa.org.ng  
www.icdfa.edu.ng.icdfa.org.ng  
aivtic.org.ng.icdfa.org.ng  
www.aivtic.org.ng.icdfa.org.ng  
gidsae.org.ng.icdfa.org.ng  
www.gidsae.org.ng.icdfa.org.ng  
icdf.org.ng.icdfa.org.ng  
www.icdf.org.ng.icdfa.org.ng  
portal.icdfa.org.ng  
www.portal.icdfa.org.ng
```

- Commands used and Output screenshots for knockpy

```
(raph@kali)-[~]
```

```
$ knockpy -d icdfa.edu.ng
```

```
icdfa.edu.ng ['131.153.147.186']
```

```
http [None, None, None, None]
```

```
https [200, None, 'Apache', 127385]
```

```
cert [True, '2026-02-27', '*.icdfa.edu.ng', ['TLS 1.2', 'TLS 1.3']]
```

```
1 domains in 00:00:09
```

```
(raph@kali)-[~]
$ knockpy -d nasarawastate.gov.ng

nasarawastate.gov.ng ['181.215.243.156']
http  [None, None, None, None]
https [None, None, None, None]
cert  [None, None, None, None]

1 domains in 00:00:06
```

```
(raph@kali)-[~]
$ knockpy -d icdfa.org.ng

icdfa.org.ng ['131.153.147.186']
http  [301, 'https://icdfa.org.ng/', 'Apache', 269]
https [302, 'https://icdfa.edu.ng', 'Apache', 0]
cert  [True, '2026-02-27', '*.icdfa.edu.ng', ['TLS 1.2', 'TLS 1.3']]

1 domains in 00:00:05
```

- **Short explanation**

Sublist3r usually finds more subdomains in a short time because it uses multiple search engines and online sources. It is fast and effective for initial subdomain enumeration.

KnockPy often finds fewer subdomains, but the results are more focused because it relies mainly on brute-force techniques using wordlists. It is slower but can discover subdomains that are not indexed by search engines.

In summary, Sublist3r is better for quick and broad discovery, while KnockPy is useful for deeper enumeration of hidden subdomains.

Part 3: Directory Enumeration with Dirb

Task

- **Dirb command**

```
(raph@kali)-[~]  
$ dirb http://192.168.146.129  
  
_____  
DIRB v2.22  
By The Dark Raver  
_____  
  
START_TIME: Sat Dec 27 18:00:43 2025  
URL_BASE: http://192.168.146.129/  
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  
  
_____  
  
GENERATED WORDS: 4612  
  
—— Scanning URL: http://192.168.146.129/ ——  
+ http://192.168.146.129/cgi-bin/ (CODE:403|SIZE:296)  
=> DIRECTORY: http://192.168.146.129/dav/  
+ http://192.168.146.129/index (CODE:200|SIZE:891)
```

- Screenshot of discovered directories

GENERATED WORDS: 4612

```
--- Scanning URL: http://192.168.146.129/ ---
+ http://192.168.146.129/cgi-bin/ (CODE:403|SIZE:296)
=> DIRECTORY: http://192.168.146.129/dav/
+ http://192.168.146.129/index (CODE:200|SIZE:891)
+ http://192.168.146.129/index.php (CODE:200|SIZE:891)
+ http://192.168.146.129/phpinfo (CODE:200|SIZE:48107)
+ http://192.168.146.129/phpinfo.php (CODE:200|SIZE:48119)
=> DIRECTORY: http://192.168.146.129/phpMyAdmin/
+ http://192.168.146.129/server-status (CODE:403|SIZE:301)
=> DIRECTORY: http://192.168.146.129/test/
=> DIRECTORY: http://192.168.146.129/twiki/
```

```
--- Entering directory: http://192.168.146.129/dav/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)
```

```
--- Entering directory: http://192.168.146.129/phpMyAdmin/ ---
+ http://192.168.146.129/phpMyAdmin/calendar (CODE:200|SIZE:4145)
+ http://192.168.146.129/phpMyAdmin/changelog (CODE:200|SIZE:74593)
+ http://192.168.146.129/phpMyAdmin/ChangeLog (CODE:200|SIZE:40540)
=> DIRECTORY: http://192.168.146.129/phpMyAdmin/contrib/
+ http://192.168.146.129/phpMyAdmin/docs (CODE:200|SIZE:4583)
+ http://192.168.146.129/phpMyAdmin/error (CODE:200|SIZE:1063)
+ http://192.168.146.129/phpMyAdmin/export (CODE:200|SIZE:4145)
+ http://192.168.146.129/phpMyAdmin/favicon.ico (CODE:200|SIZE:18902)
+ http://192.168.146.129/phpMyAdmin/import (CODE:200|SIZE:4145)
+ http://192.168.146.129/phpMyAdmin/index (CODE:200|SIZE:4145)
+ http://192.168.146.129/phpMyAdmin/index.php (CODE:200|SIZE:4145)
=> DIRECTORY: http://192.168.146.129/phpMyAdmin/js/
=> DIRECTORY: http://192.168.146.129/phpMyAdmin/lang/
```

- **Brief explanation of findings**

The Dirb scan identified several accessible directories on the target web server. These directories may expose sensitive areas such as admin panels, file upload locations, or configuration pages. Such findings are important because attackers can use them to further explore vulnerabilities like weak authentication or file inclusion issues.

Part 4: MySQL Login and Database Exploration

Task

- **MySQL login command**

```

(raph@kali)-[~]
$ mysql -u root -h 192.168.146.129 --skip-ssl
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.0.51a-3ubuntu5 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement
.

```

- Database listing

```

MySQL [(none)]> show databases
→
→ ^C
MySQL [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| dwwa |
| metasploit |
| mysql |
| owasp10 |
| tikiwiki |
| tikiwiki195 |
+-----+
7 rows in set (0.001 sec)

```

- Table listing screenshots

```

MySQL [owasp10]> show tables;
+-----+
| Tables_in_owasp10 |
+-----+
| accounts |
| blogs_table |
| captured_data |
| credit_cards |
| hitlog |
| pen_test_tools |
+-----+
6 rows in set (0.001 sec)

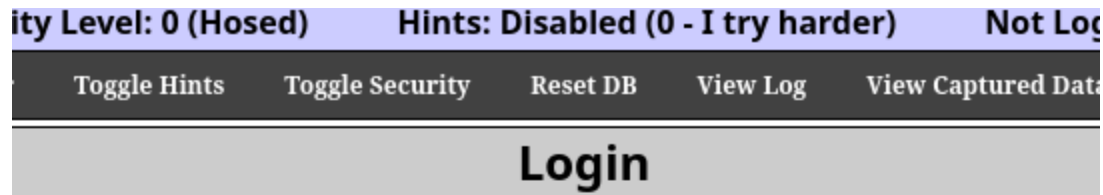
MySQL [owasp10]> █

```

Part 5: First-Order SQL Injection in Mutillidae

Task

- Injection payloads



Please sign-in

Name

usman' OR 1=1#

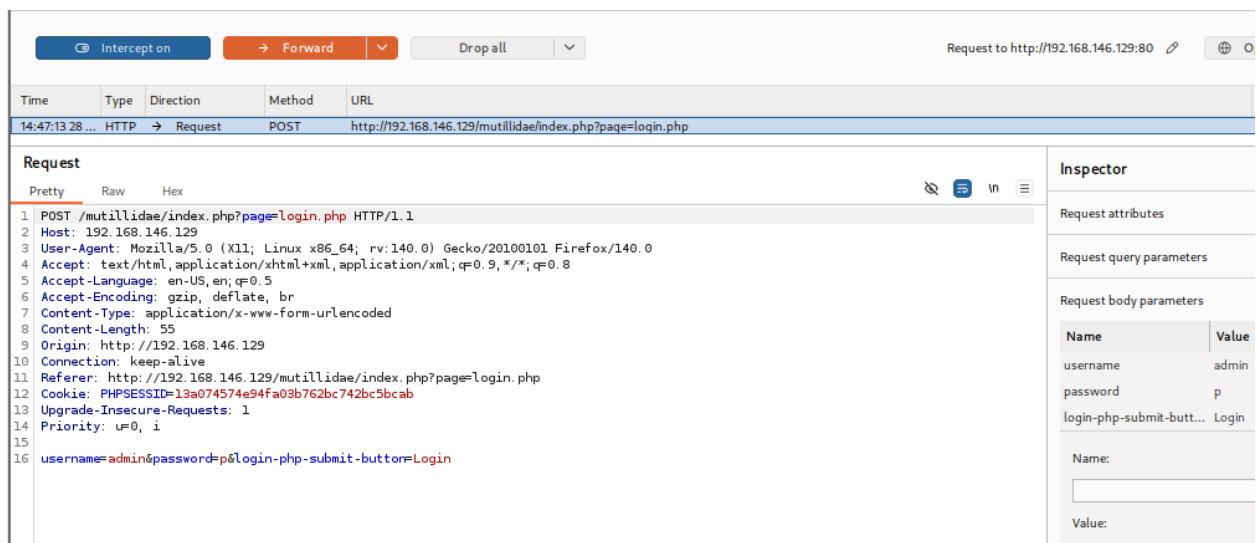
Password

...

Login

Dont have an account? [Please register here](#)

- Burp Suite screenshots



- Successful exploitation proof

2 3 4 | [Icons] EN [Speaker]

mn Vulnerable V × phpMyAdmin × 192.168.146.129/m × FoxyProxy Options × +

Not Secure http://192.168.146.129/mutillidae/index.php?page=login.php ☆

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB Object not found



Mutillidae: Born to be Hacked

Security Level: 1 (Arrogant) Hints: Disabled (0 - I try harder) Logged In Admin: admin

Home Logout Toggle Security Reset DB View Log View Captured Data

Login

 Back

You are logged in as admin

Logout

Security Level: 0 (Hosed)		Hints: Disabled (0 - I try harder)		Logged In As		
Home	Logout	Toggle Hints	Toggle Security	Reset DB	View Log	View Captured D
Login						



Back

You are logged in as admin

Logout

Part 6: POST vs GET Requests

Task

- **Short explanation of POST and GET Request**

- i. **POST** - A POST request is used when a user wants to send data to the server for processing, usually to create or update something.

Practical way to think about it:

When you log into a website or submit a registration form, your username and password are sent using a POST request.

- ii. **GET** - A GET request is used when a user wants to retrieve information from a server without changing anything on the server.

Practical way to think about it:

When you search for something on Google, the search term appears in the URL. That search is sent using a GET request.

- **Differentiation between POST and GET Requests**

A **GET request** is like asking a website for information. When you type something into a search bar and hit enter, the details of your search are added to the web address (URL), and the site sends back the results. Everything you asked for is visible in that link. A **POST request**, on the other hand, is like quietly handing information to the website. Instead of showing it in the URL, the data is sent in the background, hidden in the request body. That's why POST is used when you're logging in, submitting forms, or sending sensitive details — it keeps things more private and can handle bigger amounts of data.

Part 7: Information Gathering in Web Application Security

Task

Introduction

In cybersecurity, information gathering is one of the most important steps. It means collecting details about a system, network, or organization before taking any action. From my understanding, this stage is like doing research before writing an exam—you need to know what you're dealing with before you can succeed.

Why It Matters

Information gathering helps both attackers and defenders, but in very different ways. For attackers, it's the first step to finding weaknesses. They use it to learn about open ports, weak passwords, or outdated software that they can exploit. Without this knowledge, attackers would just be guessing, and their chances of breaking in would be low.

For defenders, information gathering is equally important. Security teams use it to understand their own systems better, identify possible risks, and prepare defenses. By knowing what attackers might see, defenders can fix vulnerabilities before they are exploited. It's like checking your house for unlocked doors and broken windows before a thief notices them.

Conclusion

In my own words, information gathering is essential because it sets the stage for everything that happens in cybersecurity. Attackers rely on it to plan their attacks, while defenders depend on it to strengthen their protection. Without proper information, both sides would be blind, and the battle between security and threats would be based on luck instead of knowledge.

Part 8: How Websites Work (Request–Response Model)

Task

1. How a website works in terms of request and response

When you visit a website, your browser (like Chrome or Edge) sends a **request** to the web server asking for a page. The server then processes that request and sends back a **response**, which usually contains the HTML, CSS, and JavaScript that make up the page. Your browser reads this response and displays it as the website you see. In short: browser asks → server answers → browser shows the page.

2. What happens when a domain name is typed into a browser

When you type something like www.example.com into the browser, the computer needs to know the actual **IP address** of the server hosting that site. Since humans use names (domains) and computers use numbers (IP addresses), the browser first checks with the DNS system to translate the domain name into its IP address. Once the IP is found, the browser sends a request to that server, and the server responds with the website content.

3. The role of the DNS server

The **DNS server** acts like the internet's phonebook. Its job is to take the domain name you typed (like google.com) and find the matching IP address of the server where the site is hosted. Without DNS, you'd have to remember long strings of numbers instead of simple names. So DNS makes the web user-friendly by connecting names to addresses.

In my own words: visiting a website is basically a conversation. You type a name, DNS finds the address, your browser asks the server for the page, and the server replies with the content.

Part 9: Role of Burp Suite in Web Application Penetration Testing

Task

- **Intercepting requests**

This means catching the communication between a user's browser and the website before it reaches the server. Security testers do this to see what kind of data is being sent and received. It's like pausing a conversation so you can listen carefully to what's being said.

- **Modifying parameters**

Once a request is intercepted, testers can change the values being sent (like usernames, IDs, or hidden form data) to see how the website reacts. It's basically like tweaking the answers on a form to test if the system accepts or rejects them.

- **Testing vulnerabilities**

By intercepting and modifying requests, testers can check if the website has weak points. For example, they might test if the site is vulnerable to SQL injection, cross-site scripting, or broken authentication. This is how they figure out where attackers could break in.

- **Why it is important for security testers**

Doing all this is important because it helps security testers think like attackers. If they can find flaws before hackers do, they can fix them and make the system stronger. In simple terms, it's like checking your house for unlocked doors and windows before a thief tries to sneak in.

In my own understanding: intercepting requests, changing parameters, and testing vulnerabilities are not about causing harm- they're about learning how attackers might exploit a system so defenders can stay one step ahead.

Conclusion

This lab helped me understand how web application reconnaissance and vulnerability discovery are carried out in real environments. By using tools like WhatWeb, Sublist3r, KnockPy, and Dirb, I learned how attackers and security testers gather information about a target before

attempting any attack. These tools made it easier to identify technologies, subdomains, and hidden directories that could expose security weaknesses.

Exploring the MySQL database on Metasploitable showed how poor security configurations and weak access controls can allow unauthorized users to view sensitive information. The SQL injection tests on Mutillidae further demonstrated how improper input validation can be exploited, especially when security settings are low. Using Burp Suite to intercept and modify requests helped me understand how attacks can still be performed even when some security measures are in place.

Overall, this lab improved my practical skills in web application security and gave me a clearer understanding of how attackers think and how defenders can protect systems. It also emphasized the importance of proper configuration, secure coding practices, and regular security testing to reduce vulnerabilities in web applications.

References

-  OWASP Foundation. *OWASP Top 10 Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/>
-  International Cybersecurity and Digital Forensics Academy. (n.d.). *ICDFA official website*.
<https://icdfa.edu.ng/>
-  WhatWeb Project. *WhatWeb – Next Generation Web Scanner*.
<https://github.com/urbanadventurer/WhatWeb>
-  Kali Linux Documentation. *Dirb – Web Content Scanner*.
<https://www.kali.org/tools/dirb/>